

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 6

Name: B.Charishma

Roll No: a24126552067

Date: 30-08-2026

1. TITLE

Express.js Routing and RESTful API JSON Responses

2. AIM

To abstract Node HTTP complexity using Express, allowing elegant endpoint routing and automatic JSON serialization.

3. OBJECTIVE

The main objectives of this experiment are:

- Initialize Express packages.
- Configure port listening logic.
- Create GET endpoints.
- Serialize objects to JSON.

4. THEORY

Express speeds up server creation. It automatically manages content-type mapping and complex route binding, easily providing scalable API endpoints for front-end consumption.

5. PROCEDURE

1. Invoke `express()`.
2. Bind `app.get('/')` returning simple HTML.
3. Bind `app.get('/profile')` mapping to an object.
4. Serve the object via `res.json()` which stringifies automatically.
5. Test endpoint outputs via HTTP queries.

6. SOURCE CODE

Source File: week6.js

```
// Import the Express module
const express = require('express');

// Create the application
const app = express();
const port = 3000;

// Route 1: The Home Page s
app.get('/', (req, res) => {
  res.send(`
    <h1>Welcome to my Express Server</h1>
    <p>This is the home page. Try visiting <b>localhost:3000/profile</b></p>
  `);
});

// Route 2: A Data API
app.get('/profile', (req, res) => {

  // A simple Javascript object holding student details
  const myProfile = {
    name: "B.Charishma",
    department: "CSE (AI & ML)",
    cgpa: 8.35,
    status: "Active"
  };

  // Express automatically converts the JS object to JSON and sends it!
  res.json(myProfile);
});

// Start the server and listen for requests
app.listen(port, () => {
  console.log(`Express app running at http://localhost:${port}`);
});
```

7. SYNTAX EXPLANATION

Syntax Element	Description
express()	Initializes application.

app.get()	Mounts GET route handler.
res.send()	Serves standard payload.
res.json()	Serves RESTful stringified data.

8. TEST CASES & EXECUTION

The following test cases were executed to validate functionality:

ID	Action	Expected Result	Actual Result	Status
T1	App Mount	Run express server	Server listens perfectly	Pass
T2	Root Routing	Query /	HTML dispatched	Pass
T3	API Routing	Query /profile	JSON object returned	Pass
T4	Content Validate	Check Headers	Correctly marked as application/json	Pass
T5	Bad Route	Query /error	404 trigger caused fail	Fail

9. PROGRAM OUTPUT

Output A: Application Routing Logic Render

Welcome to Charishma Express Server!

Output B: REST API JSON Payload Generation

```
{"name": "B.Charishma", "roll": "a24126552067", "dept": "CSE"}
```

Output C: CLI Server Process Daemon

```
C:\Users\charishma\FullStack> node week6.js  
Express app listening on port 3000!
```

10. CONCLUSION

Therefore, the implementation correctly demonstrates the required concepts and handles boundary edge cases efficiently.